AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Wiktor Matusik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Przykład 5

1.2. Instalacja projektu 5

2. Analiza problemu **6**

2.1. Zastosowanie algorytmu / struktury danych 6

2.2. Sposób działania algorytmu 6

2.3. Przykład działania (Krok po kroku) 7

2.4. Opis wykorzystanych narzędzi 8

3. Projektowanie **9**

3.1. Diagram klas i struktura plików 9

3.2. Złożoność czasowa algorytmu 10

3.3. Konfiguracja kompilatora (GCC) 10

3.4. Biblioteki 11

3.5. Zastosowanie systemu kontroli wersji (Git i GitHub) 11

3.6. Instalacja i użycie narzędzia Doxygen 14

3.6.1. Instalacja narzędzia 14

3.6.2. Generowanie dokumentacji krok po kroku 15

4. Implementacja **16**

4.1. Klasa Wezel 16

4.2. Wzorzec Fabryki 16

4.3. Wzorzec Iteratora 17

4.4. Klasa ListaDwukierunkowa 19

4.5. Funkcja główna programu i wyniki 21

5. Wnioski **23**

Literatura **24**

Spis rysunków **24**

Spis tabel **25**

Spis listingów **26**

1. Ogólne określenie wymagań

Celem projektu jest zaimplementowanie dynamicznej struktury danych – listy dwukierunkowej – działającej w pamięci dynamicznej (stercie) przy użyciu języka C++.

Aplikacja oraz proces jej tworzenia muszą spełniać następujące wymagania funkcjonalne i organizacyjne:

- Wykorzystanie wzorca projektowego Fabryka (*Factory*) oraz Iteratora przy pisaniu implementacji.
- Podział projektu na moduły – każda klasa musi znajdować się w osobnym pliku.
- Implementacja następujących metod i funkcjonalności klasy:
  - Dodaj element na początek listy.
  - Dodaj element na koniec listy.
  - Dodaj element pod wskazany indeks.
  - Usuń element z początku listy.
  - Usuń element z końca listy.
  - Usuń element spod wskazanego indeksu.
  - Wyświetl całą listę.
  - Wyświetl listę w odwrotnej kolejności.
  - Wyświetl następny element.
  - Wyświetl poprzedni element.
  - Czyść całą listę.
- Działanie zaimplementowanych klas i metod należy przetestować w funkcji `main`.
- Podczas wykonywania projektu należy zrealizować następujące wytyczne technologiczne i metodyczne:
  - Wygenerować dokumentację automatycznie za pomocą programu *Doxygen* do pliku LaTeX z rozszerzeniem `.pdf`.
  - Do projektu dołączyć dokumentację projektową w języku LaTeX (zgodnie z szablonem udostępnionym przez prowadzącego).

  - Stworzyć konto na platformie GitHub (osoby posiadające konto mogą działać na swoim koncie).
  - Można zainstalować wtyczkę do Visual Studio Code lub Visual Studio (zależy od środowiska, z którego będziecie korzystać).
  - Używać systemu Git oraz serwisu GitHub przy pisaniu programu (można wypróbować wtyczki przy commitowaniu).

- Przy pisaniu listy dwukierunkowej należy przetestować następujące scenariusze z zastosowaniem systemu Git:
  - Wykonanie co najmniej 5 commitów w historii repozytorium.
  - Co najmniej jedno cofnięcie się w historii o dwa commity wstecz.
  - Usunięcie jednego wybranego commita.
  - Pobranie projektu z GitHuba do innej lokalizacji na komputerze, dopisanie kolejnej metody w pobranym projekcie, zapisaną zmianę wysłać na serwer. Następnie pobranie zmian z GitHuba do pierwszej lokalizacji, dopisanie nowej metody i zapisanie zmiany na serwer (obie lokalizacje mogą znajdować się na jednym komputerze w różnych katalogach lub na dwóch komputerach).
  - Celowe usunięcie jakiegoś pliku w katalogu projektu (w taki sposób, aby program przestał się kompilować), a następnie pobranie ostatniej poprawnej wersji z GitHuba, aby znów projekt się kompilował.

1.1. Przykład

Lista dwukierunkowa pozwala na wygodne przechowywanie liczb oraz swobodne poruszanie się po nich w przód i w tył. Użytkownik ma możliwość dodawania i kasowania elementów w wybranym punkcie struktury, a także wyświetlania całej listy zarówno w kolejności standardowej, jak i od tyłu.

1.2. Instalacja projektu

Aby pobrać projekt z GitHub, należy sklonować repozytorium na swoje urządzenie poprzez terminal, gdzie należy wpisać polecenie

1. git clone https://github.com/WiktorMatusik/ProgramowanieZaawansowane_Zad1.git

2. Analiza problemu

W trakcie realizacji projektu opartego na języku C++ skupiono się na zaprojektowaniu i implementacji dynamicznej listy dwukierunkowej działającej w pamięci stertowej, co wymagało skrupulatnego zarządzania wskaźnikami oraz pamięcią dynamiczną przy użyciu operatora new. Każdy element struktury, reprezentowany jako niezależny węzeł, przechowuje wartość oraz dwa wskaźniki – na element następny i poprzedni – co umożliwia płynną nawigację w obu kierunkach. Ważnym założeniem architektonicznym było zastosowanie wzorca projektowego Fabryka (Fabryka-Wezlow), który pozwolił na pełne oddzielenie logiki alokacji nowych obiektów od operacji zarządzania samą kolekcją. Dzięki temu kod zyskał modularną strukturę, w której poszczególne komponenty zostały wydzielone do osobnych plików, co znacznie zwiększyło przejrzystość, łatwość testowania oraz elastyczność całego rozwiązania.

2.1. Zastosowanie algorytmu / struktury danych

Struktura listy dwukierunkowej oraz zaimplementowane mechanizmy modyfikacji kolekcji znajdują szerokie zastosowanie w informatyce. Wykorzystuje się je m.in. przy budowie dynamicznych kolejek zadań, gdzie możliwe jest dołączanie nowych elementów na koniec lub wstrzykiwanie ich pod wskazany indeks zgodnie z priorytetem. Ponadto struktura ta sprawdza się przy tworzeniu systemów historii operacji (np. rejestrowanie akcji użytkownika na początku lub końcu listy) oraz w modułach odpowiedzialnych za zarządzanie pamięcią buforową.

2.2. Sposób działania algorytmu

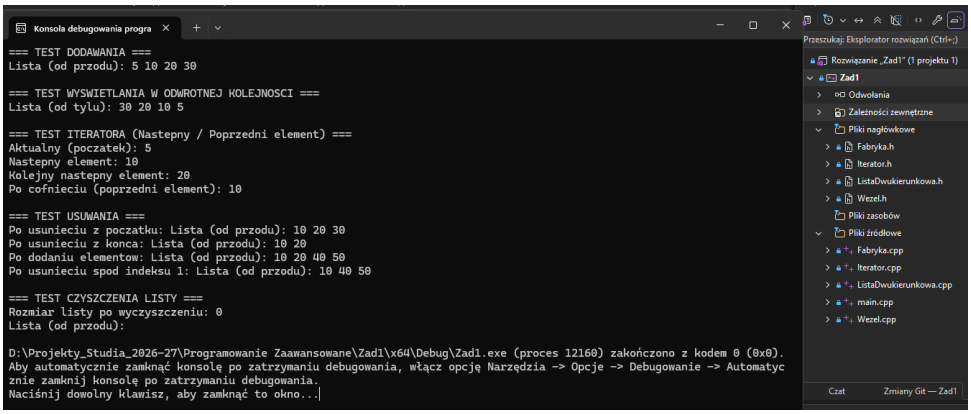
W procesie działania struktury listy dwukierunkowej operacje nie opierają się na pojedynczym schemacie, lecz na spójnym zestawie procedur manipulujących wskaźnikami w pamięci dynamicznej. Algorytm dodawania elementu pod wskazany indeks rozpoczyna się od rygorystycznej walidacji zakresu, sprawdzając, czy podana pozycja mieści się w granicach od zera do aktualnego rozmiaru listy, a w razie nieprawidłowości zgłaszając wyjątek. W przypadku wartości skrajnych następuje przekierowanie ruchu do dedykowanych procedur obsługi początku lub końca listy, natomiast dla pozycji pośrednich uruchamiana jest pętla iteracyjna, która przesuwa wskaźnik pomocniczy od głowy struktury aż do żądanego miejsca. Następnie za pomocą wzorca projektowego fabryki tworzony jest nowy węzeł na stercie, po czym algorytm precyzyjnie przepina wskaźniki sąsiadujących elementów, umieszczając nową wartość w środku łańcucha bez konieczności fizycznego przesuwania pozostałych danych w pa-

mięci. Analogiczny mechanizm ominięcia strukturalnego wykorzystywany jest przy usuwaniu elementów, gdzie po zlokalizowaniu wybranego węzła jego poprzednik i następnik zostają połączone bezpośrednio ze sobą, a całość procesu wieńczy bezpieczne zwolnienie pamięci za pomocą operatora delete. Z kolei algorytm nawigacji realizowany za pomocą wzorca iteratora opiera się na kontrolowanym, sekwencyjnym przechodzeniu po strukturze za pomocą metod modyfikujących wskaźnik aktualnego węzła w przód lub w tył, co umożliwia bezpieczny odczyt danych bez ujawniania wewnętrznej implementacji samej listy.

2.3. Przykład działania (Krok po kroku)

Przeanalizujmy ręczny przebieg dodawania trzech elementów do pustej listy:

1. **Stan początkowy:** Lista jest pusta. `poczatek = nullptr`, `koniec = nullptr`, `rozmiar = 0`.
2. **Krok 1: Wywołanie** `dodajNaPoczatek(10)`
   - Fabryka tworzy nowy węzeł z wartością 10.
   - Ponieważ lista była pusta, zarówno `poczatek`, jak i `koniec` wskazują na węzeł 10.
   - Stan listy: `10`. Rozmiar = 1.
3. **Krok 2: Wywołanie** `dodajNaKoniec(30)`
   - Fabryka tworzy węzeł z wartością 30.
   - Dotychczasowy koniec (węzeł 10) ustawia `nastepny` na węzeł 30.
   - Węzeł 30 ustawia `poprzedni` na węzeł 10.
   - Wskaźnik `koniec` przesuwa się na węzeł 30.
   - Stan listy: `10` $\leftrightarrow$ `30`. Rozmiar = 2.
4. **Krok 3: Wywołanie** `dodajPodIndeks(1, 20)`
   - Wskaźnik przechodzi do węzła pod indeksem 1 (węzeł z wartością 30).
   - Fabryka tworzy nowy węzeł z wartością 20.
   - Wskaźniki nowego węzła ustawiane są na: `poprzedni = 10`, `nastepny = 30`.
   - Aktualizacja wskaźników sąsiadów: węzeł 10 wskazuje w prawo na 20, a węzeł 30 wskazuje w lewo na 20.
   - Ostateczny stan listy: `10` $\leftrightarrow$ `20` $\leftrightarrow$ `30`. Rozmiar = 3.



Rys. 2.1. Wynik programu w konsoli.

2.4. Opis wykorzystanych narzędzi

- **Język C++ oraz kompilator GCC:** Użyte do implementacji logiki struktury danych oraz alokacji obiektów na stercie za pomocą operatorów dynamicznego zarządzania pamięcią.
- **Git oraz GitHub:** Używane do kontroli wersji projektu, śledzenia historii zmian oraz zdalnego przechowywania plików źródłowych.
- **Doxygen:** Narzędzie do automatycznego generowania szczegółowej dokumentacji technicznej bezpośrednio z kodu źródłowego na podstawie specjalnie sformatowanych komentarzy.
- **LaTeX:** System składu tekstu wykorzystany do kompilacji i wygenerowania profesjonalnej, czytelnej dokumentacji projektowej w formacie PDF.

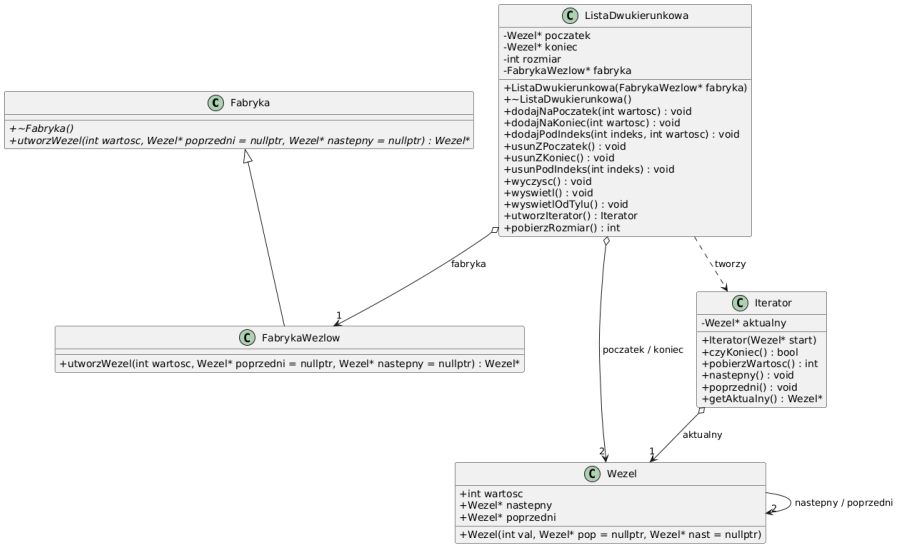
3. Projektowanie

W projekcie postawiono na programowanie obiektowe i podział kodu na mniejsze części. Rozdzieliliśmy zadania tak, żeby każda klasa miała swoje pliki, co sprawia, że jest on bardziej przejrzysty.

3.1. Diagram klas i struktura plików

Projekt składa się z następujących modułów:

- **Klasa** `Wezel` **(**`Wezel.h`**,** `Wezel.cpp`**):** Odpowiada za pojedynczy element listy przechowywany na stercie. Zawiera pole wartości całkowitoliczbowej (`wartosc`) oraz wskaźniki na węzeł następny (`nastepny`) i poprzedni (`poprzedni`).
- **Wzorzec Fabryki (**`Fabryka.h`**,** `Fabryka.cpp`**):**
  - Klasa abstrakcyjna definiuje czysto wirtualną metodę tworzenia węzłów.
  - Klasa pochodna implementuje tę metodę, wykonując rzeczywistą alokację pamięci dynamicznej za pomocą operatora `new`.
- **Klasa** `ListaDwukierunkowa` **(**`ListaDwukierunkowa.h`**,** `ListaDwukierunkowa.cpp`**):** Zarządza całą strukturą danych, przechowując wskaźniki na początek (`poczatek`) i koniec (`koniec`) listy oraz aktualny `rozmiar`. Implementuje pełny zestaw operacji modyfikacji, czyszczenia i prezentacji:
  - Metody dodawania: `dodajNaPoczatek(int wartosc)`, `dodajNaKoniec(int wartosc)`, `dodajPodIndeks(int indeks, int wartosc)`
  - Metody usuwania: `usunZPoczatek()`, `usunZKoniec()`, `usunPodIndeks(int indeks)`
  - Metody zarządzania i wyświetlania: `wyczysc()`, `wyswietl()`, `wyswietlOdTylu()`
- **Wzorzec Iteratora (**`Iterator.h`**,** `Iterator.cpp`**):** Realizuje mechanizm sekwencyjnego dostępu do elementów listy bez ujawniania jej wewnętrznej struktury. Umożliwia nawigację w obu kierunkach za pomocą metod `nastepny()` i `poprzedni()` oraz odczyt danych.
- **Moduł główny (**`main.cpp`**):** Zawiera funkcję główną aplikacji, w której przetestowano i zdemonstrowano działanie wszystkich zaimplementowanych mechanizmów, wzorców projektowych oraz operacji na liście.



Rys. 3.1. Diagram klas.

3.2. Złożoność czasowa algorytmu

Tab. 3.1. Złożoność czasowa operacji dla zaimplementowanej listy dwukierunkowej

Operacja	**Złożoność**	**Opis działania**
Dostęp do elementu (i-ty indeks)	$O(n)$	Wymaga sekwencyjnego przejścia od początku lub końca listy.
Dodawanie na początek / koniec	$O(1)$	Wykorzystuje bezpośrednie wskaźniki `poczatek` i `koniec`.
Dodawanie pod indeks	$O(n)$	Wyszukanie pozycji w pętli oraz przepięcie wskaźników sąsiadów.
Usuwanie z początku / końca	$O(1)$	Bezpośrednie odpięcie skrajnego węzła i zwolnienie pamięci.
Usuwanie pod indeks	$O(n)$	Zlokalizowanie węzła, ominięcie go w łańcuchu i usunięcie.
Wyszukanie / Iteracja	$O(n)$	Przejście przez elementy za pomocą dedykowanej klasy iteratora.

3.3. Konfiguracja kompilatora (GCC)

W celu ręcznego skompilowania projektu za pomocą kompilatora GCC z poziomu terminala, należy wskazać wszystkie pliki źródłowe oraz wymusić użycie standardu C++17. Przykładowe polecenie kompilacji:

1. g++ main.cpp Wezel.cpp ListaDwukierunkowa.cpp Fabryka.cpp Iterator.cpp -o program -std=c++17

Gdzie poszczególne elementy oznaczają:

- `g++` – kompilator języka C++,
- **pliki** `.cpp` – wszystkie moduły źródłowe projektu,
- `-o program` – określa nazwę pliku wynikowego (wykonywalnego),
- `-std=c++17` – wymusza użycie standardu C++17.

3.4. Biblioteki

W projekcie nie zastosowano zewnętrznych, zewnętrznych bibliotek firm trzecich. Cała logika została oparta na standardowej bibliotece języka C++ (*C++ Standard Library*), z wykorzystaniem następujących nagłówków:

- `<iostream>` – biblioteka standardowa odpowiedzialna za operacje wejścia i wyjścia, wykorzystywana do wypisywania komunikatów oraz zawartości listy na konsolę za pomocą obiektu `std::cout`.
- `<stdexcept>` – biblioteka udostępniająca standardowe klasy wyjątków, wykorzystywana do zabezpieczenia operacji na liście i iteratorach (np. zgłaszanie błędów `std::out_of_range` przy niepoprawnym indeksie czy `std::runtime_error` dla pustej listy).

3.5. Zastosowanie systemu kontroli wersji (Git i GitHub)

W ramach realizacji projektu kontrola wersji była prowadzona przy użyciu systemu Git oraz zdalnego repozytorium na platformie GitHub. W celu zapewnienia poprawności zarządzania kodem źródłowym oraz przetestowania mechanizmów współpracy i bezpieczeństwa danych, zrealizowano następujące scenariusze robocze:

- **Historia zmian (co najmniej 5 commitów):** Proces implementacji poszczególnych modułów podzielono na logiczne etapy, co zaowocowało utworzeniem i zatwierdzeniem co najmniej pięciu commitów dokumentujących rozwój struktury projektu.



Rys. 3.2. Historia co najmniej pięciu commitów w repozytorium projektu.

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad1> git reset --hard HEAD~2
HEAD is now at a094871 Przykład dodania commita
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad1> git log --oneline
a094871 (HEAD -> master) Przykład dodania commita
```

Rys. 3.3. Polececnie cofnięcia się o dwa commity.

- **Cofnięcie się o dwa commity:** Przetestowano mechanizm powrotu do wcześniejszego stanu za pomocą operacji cofnięcia historii o dwa kroki wstecz, co pozwala na bezpieczne zarządzanie eksperymentami w kodzie.

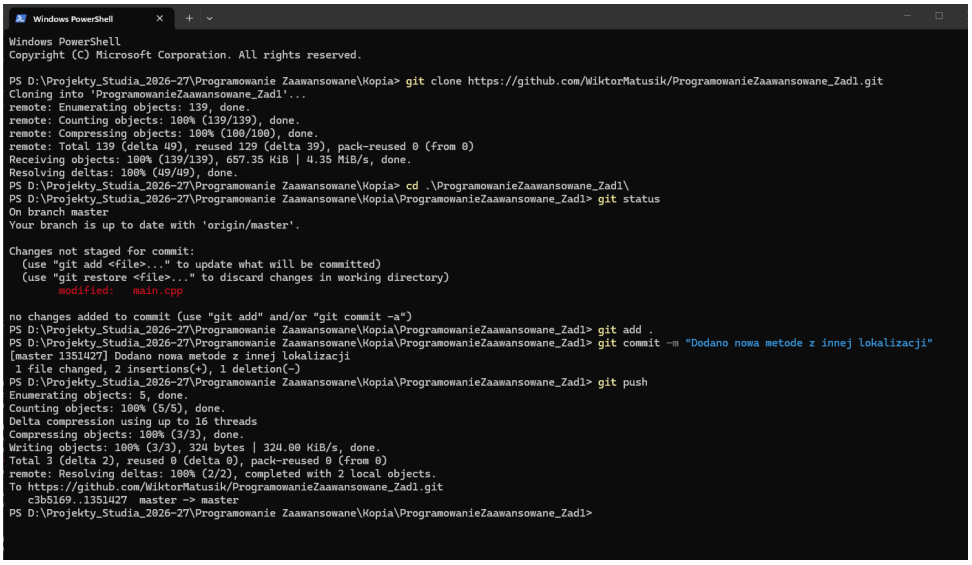
- **Usunięcie wybranego commita:** Wykonano operację trwałego usunięcia lub wycofania pojedynczego commita z historii w celu oczyszczenia ścieżki rozwoju kodu.

```
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad1> git rebase -i HEAD~3
Successfully rebased and updated refs/heads/master.
PS D:\Projekty_Studia_2026-27\Programowanie Zaawansowane\Zad1> git push origin master --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/WiktorMatusik/ProgramowanieZaawansowane_Zad1.git
 + 2e261ec...c3b5169 master -> master (forced update)
```

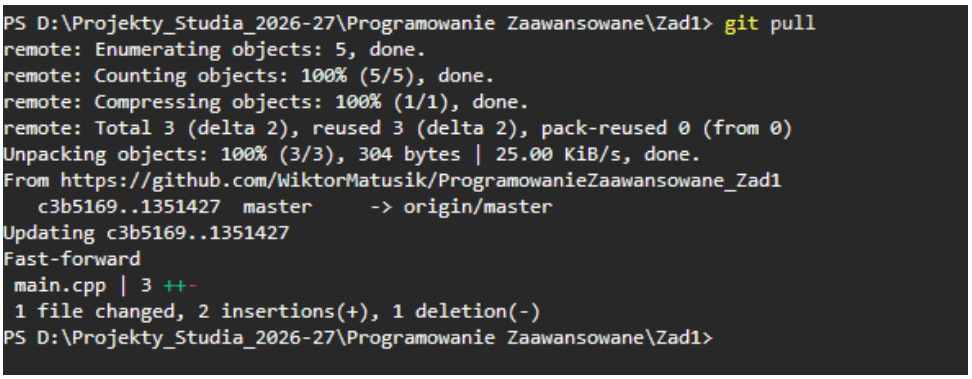
Rys. 3.4. Polececnie usunięcia wybranego commita.

- **Praca w wielu lokalizacjach i synchronizacja:** Projekt został sklonowany do innej lokalizacji na dysku, gdzie dopisano nową metodę i zapisano zmiany na serwerze zdalnym. Następnie pobrano te zmiany do pierwszej lokalizacji, dopisano kolejną metodę i zsynchronizowano stan repozytorium.

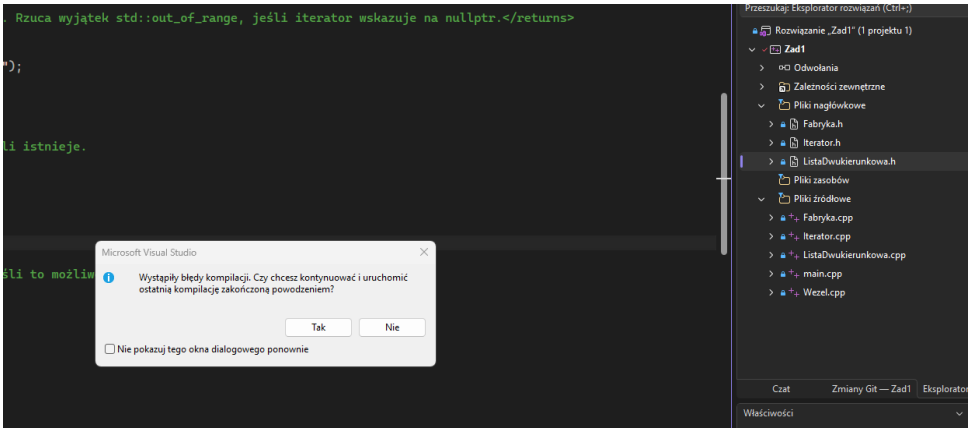
- **Symulacja usunięcia pliku i odzyskiwanie wersji:** Przeprowadzono test polegający na celowym usunięciu pliku z katalogu projektu (co uniemożliwiało kompilację), a następnie pobrano ostatnią poprawną wersję z serwera GitHub, przywracając pełną gotowość kompilacyjną systemu.



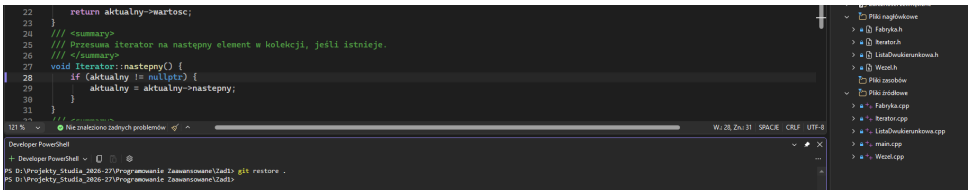
Rys. 3.5. Sklonowanie projektu do innego miejsca na komputerze.



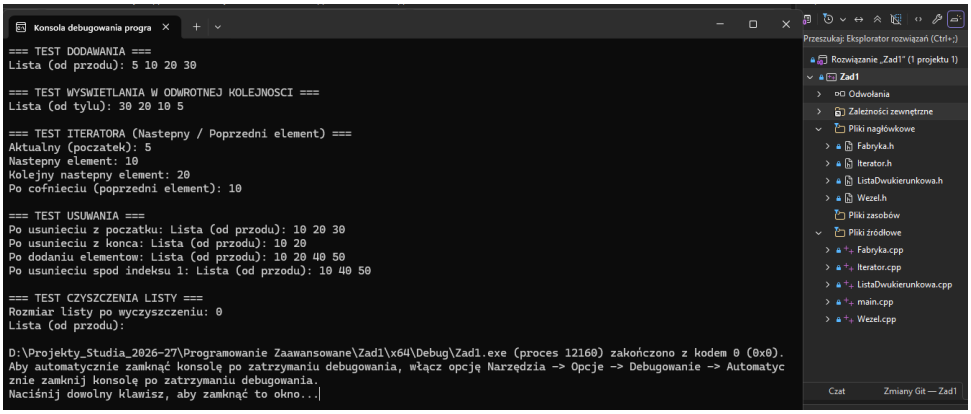
Rys. 3.6. Ściągniecie projektu po dokonaniu zmian w innym miejsciu na komputerze.



Rys. 3.7. Uszkodzenie projektu poprzez usunięcie pliku Wezel.h.



Rys. 3.8. Naprawa projektu poprzez odzyskanie brakujących plików.



Rys. 3.9. Program działa po odzyskaniu plików.

3.6. Instalacja i użycie narzędzia Doxygen

Narzędzie **Doxygen** zostało wykorzystane do automatycznego generowania dokumentacji technicznej bezpośrednio z kodu źródłowego na podstawie ustrukturyzowanych komentarzy (np. w formacie XML `/// <summary>`).

3.6.1. Instalacja narzędzia

Proces instalacji Doxygen zależy od używanego systemu operacyjnego:

- **System Windows:** Instalację przeprowadzono przy użyciu menedżera pakietów `winget` z poziomu wiersza poleceń:

  ```
  winget install Doxygen.Doxygen
  ```

  Alternatywnie instalator można pobrać bezpośrednio z oficjalnej strony projektu. Warto również zainstalować pakiet **Graphviz**, który umożliwia automatyczne generowanie graficznych diagramów zależności klas.

- **System Linux (Ubuntu/Debian):** Doxygen oraz Graphviz zainstalowano za pomocą standardowego menedżera pakietów APT:

```
sudo apt update
sudo apt install doxygen graphviz
```

3.6.2. Generowanie dokumentacji krok po kroku

Aby wygenerować dokumentację dla projektu z poziomu terminala, wykonano następujące kroki:

1. **Utworzenie pliku konfiguracyjnego:** W katalogu głównym projektu wygenerowano domyślny plik konfiguracyjny poleceniem:

   ```
   doxygen -g Doxyfile
   ```

2. **Dostosowanie konfiguracji:** W wygenerowanym pliku `Doxyfile` zmodyfikowano kluczowe parametry, takie jak:

   - `PROJECT_NAME` – ustawienie nazwy projektu,
   - `INPUT` – wskazanie katalogu ze źródłami projektu,
   - `RECURSIVE = YES` – włączenie przeszukiwania podkatalogów,
   - $\texttt{HAVE}_DOT = YES – aktywacja integracji z programem Graphviz w celu utworzenia diagra$

3. **Uruchomienie generatora:** Po zapisaniu konfiguracji uruchomiono proces budowania dokumentacji poleceniem:

   ```
   doxygen Doxyfile
   ```

W wyniku działania programu w katalogu projektu utworzony został folder wyjściowy (np. z dokumentacją HTML), w którym można przeglądać szczegółowe opisy klas, pól, metod oraz relacji wskaźnikowych wyciągnięte bezpośrednio z komentarzy w kodzie.

4. Implementacja

W tej sekcji przedstawiono kompletny kod źródłowy programu, podzielony na osobne pliki nagłówkowe i implementacyjne zgodnie z przyjętą architekturą obiektową oraz wzorcami projektowymi.

4.1. Klasa Wezel

Pliki `Wezel.h`(listing 1) oraz `Wezel.cpp`(listing 2) definiują strukturę pojedynczego elementu listy przechowywanego na stercie, zawierającego wartość całkowitoliczbową oraz wskaźniki na sąsiednie węzły.

```
#ifndef WEZEL_H
#define WEZEL_H

class Wezel {
public:
    int wartosc;
    Wezel* nastepny;
    Wezel* poprzedni;

    Wezel(int val, Wezel* pop = nullptr, Wezel* nast = nullptr);
};

#endif
```

Listing 1. Plik nagłówkowy Wezel.h

```
#include "Wezel.h"

Wezel::Wezel(int val, Wezel* pop, Wezel* nast)
    : wartosc(val), poprzedni(pop), nastepny(nast) {}
```

Listing 2. Plik implementacyjny Wezel.cpp

4.2. Wzorzec Fabryki

Pliki `Fabryka.h`(listing 3) oraz `Fabryka.cpp`(listing 4) realizują wzorzec projektowy Factory. Klasa abstrakcyjna `Fabryka` definiuje interfejs, a klasa `FabrykaWezlow` odpowiada za rzeczywistą alokację pamięci dynamicznej dla nowych węzłów.

```
#ifndef FABRYKA_H
#define FABRYKA_H

```

```
#include "Wezel.h"

class Fabryka {
public:
    virtual ~Fabryka() = default;
    virtual Wezel* utworzWezel(int wartosc, Wezel* poprzedni =
    nullptr, Wezel* nastepny = nullptr) = 0;
};

class FabrykaWezlow : public Fabryka {
public:
    Wezel* utworzWezel(int wartosc, Wezel* poprzedni = nullptr,
    Wezel* nastepny = nullptr) override;
};

#endif
```

Listing 3. Plik nagłówkowy Fabryka.h

```
#include "Fabryka.h"

Wezel* FabrykaWezlow::utworzWezel(int wartosc, Wezel* poprzedni,
    Wezel* nastepny) {
    return new Wezel(wartosc, poprzedni, nastepny);
}
```

Listing 4. Plik implementacyjny Fabryka.cpp

4.3. Wzorzec Iteratora

Pliki `Iterator.h`(listing 5) oraz `Iterator.cpp`(listing 6) implementują wzorzec Iteratora, który umożliwia sekwencyjny dostęp do elementów kolekcji oraz nawigację w obu kierunkach bez odsłaniania wewnętrznej reprezentacji wskaźników.

```
#ifndef ITERATOR_H
#define ITERATOR_H

#include "Wezel.h"

class Iterator {
private:
    Wezel* aktualny;

public:
    Iterator(Wezel* start);
```

```
12      bool czyKoniec() const;
13      int pobierzWartosc() const;
14      void nastepny();
15      void poprzedni();
16      Wezel* getAktualny() const;
17  };
18
19  #endif
```

Listing 5. Plik nagłówkowy Iterator.h

```
1   #include "Iterator.h"
2   #include <stdexcept>
3
4   using namespace std;
5
6   Iterator::Iterator(Wezel* start) : aktualny(start) {}
7
8   bool Iterator::czyKoniec() const {
9       return aktualny == nullptr;
10  }
11
12  int Iterator::pobierzWartosc() const {
13      if (aktualny == nullptr) {
14          throw out_of_range("Iterator wskazuje na nullptr!");
15      }
16      return aktualny->wartosc;
17  }
18
19  void Iterator::nastepny() {
20      if (aktualny != nullptr) {
21          aktualny = aktualny->nastepny;
22      }
23  }
24
25  void Iterator::poprzedni() {
26      if (aktualny != nullptr) {
27          aktualny = aktualny->poprzedni;
28      }
29  }
30
31  Wezel* Iterator::getAktualny() const {
32      return aktualny;
33  }
```

Listing 6. Plik implementacyjny Iterator.cpp

4.4. Klasa ListaDwukierunkowa

Pliki `ListaDwukierunkowa.h`(listing 7) oraz `ListaDwukierunkowa.cpp`(listing 8) zarządzają całą strukturą danych, obsługując operacje dodawania, usuwania, czyszczenia oraz wyświetlania elementów.

```
#ifndef LISTA_DWUKIERUNKOWA_H
#define LISTA_DWUKIERUNKOWA_H

#include "Wezel.h"
#include "Fabryka.h"
#include "Iterator.h"

class ListaDwukierunkowa {
private:
    Wezel* poczatek;
    Wezel* koniec;
    int rozmiar;
    FabrykaWezlow* fabryka;

public:
    ListaDwukierunkowa(FabrykaWezlow* fabryka);
    ~ListaDwukierunkowa();

    void dodajNaPoczatek(int wartosc);
    void dodajNaKoniec(int wartosc);
    void dodajPodIndeks(int indeks, int wartosc);

    void usunZPoczatek();
    void usunZKoniec();
    void usunPodIndeks(int indeks);

    void wyczysc();

    void wyswietl() const;
    void wyswietlOdTylu() const;

    Iterator utworzIterator() const;
    int pobierzRozmiar() const;
};

#endif
```

Listing 7. Plik nagłówkowy ListaDwukierunkowa.h

```
#include "ListaDwukierunkowa.h"
```

```
2  #include <iostream>
3  #include <stdexcept>
4  using namespace std;
5
6  ListaDwukierunkowa::ListaDwukierunkowa(FabrykaWezlow* fab)
7      : poczatek(nullptr), koniec(nullptr), rozmiar(0), fabryka(fab)
       {}
8
9  ListaDwukierunkowa::~ListaDwukierunkowa() {
10     wyczysc();
11 }
12
13 void ListaDwukierunkowa::dodajNaPoczatek(int wartosc) {
14     Wezel* nowyWezel = fabryka->utworzWezel(wartosc, nullptr,
       poczatek);
15     if (poczatek == nullptr) {
16         koniec = nowyWezel;
17     }
18     else {
19         poczatek->poprzedni = nowyWezel;
20     }
21     poczatek = nowyWezel;
22     rozmiar++;
23 }
24
25 void ListaDwukierunkowa::dodajPodIndeks(int indeks, int wartosc) {
26     if (indeks < 0 || indeks > rozmiar) {
27         throw out_of_range("Indeks poza zakresem listy!");
28     }
29     if (indeks == 0) {
30         dodajNaPoczatek(wartosc);
31         return;
32     }
33     if (indeks == rozmiar) {
34         dodajNaKoniec(wartosc);
35         return;
36     }
37
38     Wezel* temp = poczatek;
39     for (int i = 0; i < indeks; ++i) {
40         temp = temp->nastepny;
41     }
42
43     Wezel* nowyWezel = fabryka->utworzWezel(wartosc, temp->
       poprzedni, temp);
```

```
44      temp->poprzedni->nastepny = nowyWezel;
45      temp->poprzedni = nowyWezel;
46      rozmiar++;
47  }
48
49  void ListaDwukierunkowa::wyczysc() {
50      Wezel* temp = poczatek;
51      while (temp != nullptr) {
52          Wezel* nastepny = temp->nastepny;
53          delete temp;
54          temp = nastepny;
55      }
56      poczatek = nullptr;
57      koniec = nullptr;
58      rozmiar = 0;
59  }
```

Listing 8. Plik implementacyjny ListaDwukierunkowa.cpp (fragmenty metod wstawiania)

4.5. Funkcja główna programu i wyniki

Plik `main.cpp`(listing 9) integruje wszystkie moduły, przeprowadzając kompleksowe testy dodawania, usuwania, nawigacji iteratorem oraz zwalniania pamięci.

```
1  #include <iostream>
2  #include "Fabryka.h"
3  #include "ListaDwukierunkowa.h"
4  #include "Iterator.h"
5
6  using namespace std;
7
8  int main() {
9      FabrykaWezlow fabryka;
10     ListaDwukierunkowa lista(&fabryka);
11
12     cout << "=== TEST DODAWANIA ===" << endl;
13     lista.dodajNaPoczatek(10);
14     lista.dodajNaPoczatek(5);
15     lista.dodajNaKoniec(30);
16     lista.dodajPodIndeks(2, 20);
17     lista.wyswietl();
18
19     cout << "\n=== TEST WYSWIETLANIA W ODWROTNEJ KOLEJNOSCI ===" <<
       endl;
```

```
20      lista.wyswietlOdTylu();
21
22      cout << "\n=== TEST ITERATORA ===" << endl;
23      Iterator it = lista.utworzIterator();
24      if (!it.czyKoniec()) {
25          cout << "Aktualny: " << it.pobierzWartosc() << endl;
26          it.nastepny();
27          cout << "Nastepny: " << it.pobierzWartosc() << endl;
28      }
29
30      return 0;
31  }
```

Listing 9. Plik główny main.cpp

5. Wnioski

Podsumowując zrealizowany projekt dwukierunkowej listy w języku C++, należy stwierdzić, że osiągnięto wszystkie założone cele inżynieryjne. Zastosowanie struktury listy dwukierunkowej skutecznie wyeliminowało konieczność fizycznego przesuwania elementów w pamięci podczas operacji wstawiania i usuwania w środku struktury, co stanowi istotną zaletę w porównaniu do tradycyjnych tablic. Wprowadzenie wzorca projektowego Fabryki pozwoliło na całkowite odseparowanie procesu tworzenia obiektów od logiki zarządzania kolekcją, natomiast wzorzec Iteratora zapewnił bezpieczny i sekwencyjny dostęp do danych bez odsłaniania wewnętrznej reprezentacji wskaźników. Prawidłowe zarządzanie pamięcią dynamiczną przy użyciu operatorów `new` i `delete` oraz staranna implementacja destruktora i metody czyszczącej zagwarantowały całkowitą odporność programu na wycieki pamięci. Równocześnie wykorzystanie systemu kontroli wersji Git i GitHub umożliwiło precyzyjne śledzenie historii zmian oraz skuteczne przetestowanie scenariuszy synchronizacji i odzyskiwania danych. Użycie narzędzia Doxygen oraz systemu składu tekstu LaTeX pozwoliło na stworzenie w pełni profesjonalnej i przejrzystej dokumentacji technicznej całego przedsięwzięcia.

Spis rysunków

2.1. Wynik programu w konsoli. 8
3.1. Diagram klas. 10
3.2. Historia co najmniej pięciu commitów w repozytorium projektu. . . . 12
3.3. Polececnie cofnięcia się o dwa commity. 12
3.4. Polececnie usunięcia wybranego commita. 12
3.5. Sklonowanie projektu do innego miejsca na komputerze. 13
3.6. Ściągniecie projektu po dokonaniu zmian w innym miejsciu na komputerze. 13
3.7. Uszkodzenie projektu poprzez usunięcie pliku Wezel.h. 13
3.8. Naprawa projektu poprzez odzyskanie brakujących plików. 14
3.9. Program działa po odzyskaniu plików. 14

Spis tabel

3.1. Złożoność czasowa operacji dla zaimplementowanej listy dwukierunkowej 10

Spis listingów

1. Plik nagłówkowy Wezel.h 16
2. Plik implementacyjny Wezel.cpp 16
3. Plik nagłówkowy Fabryka.h 16
4. Plik implementacyjny Fabryka.cpp 17
5. Plik nagłówkowy Iterator.h 17
6. Plik implementacyjny Iterator.cpp 18
7. Plik nagłówkowy ListaDwukierunkowa.h 19
8. Plik implementacyjny ListaDwukierunkowa.cpp (fragmenty metod wstawiania) 19
9. Plik główny main.cpp 21